Как читать конспект темы 12

В демоверсии ФИПИ 2027 задание 12 использует исполнителя МТ — машину Тьюринга. Основной маршрут для текущего формата: лента, головка и состояние (§10, исходные стр.15–16) → одна команда (§10.3, стр.16–17) → ручной разбор (§11, стр.17) → смысл прохода и обратные задачи (§12, стр.17–19) → разряды и несколько фаз (§13, стр.19–20) → моделирование (§14, стр.21–23) → практика №13–20 (стр.25–26). Дополнение после исходных 27 страниц содержит недостающие маленькие разборы и вопросы с ответами. Номера страниц в этих указателях — напечатанные в исходном конспекте.

Редактор (§§2–9; практика №1–12) сохранён для исторических задач банка и изучения строковых алгоритмов. Его не требуется изучать перед МТ. Наличие раздела в банке само по себе не определяет содержание текущей демоверсии. Подробные объяснения, все исходные таблицы, программы и 20 упражнений сохранены.

В демоверсии ФИПИ 2027 определены L/R/S: запись выполняется до движения, а S останавливает уже после записи. В исходном конспекте N показано как дополнительное соглашение отдельных условий: оно означает отсутствие сдвига с продолжением работы только там, где это определено. N не включено в набор команд демоверсии ФИПИ 2027. Начальная метка q0 не означает остановку. Пустая клетка таблицы в демоверсии помечает невозможное при допустимом выполнении сочетание, а не команду остановки.

В §12.2 (исходная стр.18) рассматривается непустая строка длины L, допускающая начальные нули, при $1 \le K \le L$. Для канонической записи положительного числа без начальных нулей допустимые конструкции и границы нужно вывести заново. Случай $K = 0$ в исходном конспекте правильно разобран отдельно.

Источники сверки: ФИПИ, проекты документов ЕГЭ 2027 — https://fipi.ru/ege/demoversii-specifikacii-kodifikatory ; Яндекс Учебник, статья277 — https://education.yandex.ru/ege/inf/article/277-reshenie-zadaniia-12-mashina-tiuringa . Это содержательное ревью исходника: частоты всего банка, готовность стенда и публикационная приёмка не аттестованы.

Задание 12

Редактор
и машина Тьюринга

Понять команды. Проследить изменения.
Найти закономерность. Проверить решение.

Подробный курс по двум исполнителям с последовательным введением понятий, таблицами выполнения и программами на Python.

20 тренировочных вопросов и задач
Ответы с объяснениями
Математические способы и точное моделирование

Материал для осмысленной подготовки к ЕГЭ по информатике
15 октября 2026

Содержание

Первые главы вводят основы. После Редактора следует самостоятельный разбор машины Тьюринга. Практика и объяснения ответов расположены в конце.

1. Что именно нужно понять . . . 3

2. Редактор: из чего состоит строка . . . 3

3. Как условия управляют командами . . . 4

4. Полный пример Редактора вручную . . . 6

5. Как точно воспроизвести Редактор на Python . . . 7

6. От отдельных замен к закономерности . . . 9

7. Подсчёт замен, неизвестный порядок и маркеры . . . 11

8. Перебор неизвестного параметра . . . 12

9. Сколько различных конечных строк возможно . . . 15

10. Машина Тьюринга: лента, головка, состояние . . . 15

11. Первый полный разбор машины Тьюринга . . . 17

12. Как увидеть смысл всей машины . . . 17

13. Двоичные числа и несколько проходов по ленте . . . 19

14. Универсальное воспроизведение машины на Python . . . 21

15. Как выбрать способ решения на экзамене . . . 23

16. Практика: сначала реши самостоятельно . . . 24

17. Ответы с объяснениями . . . 25

18. Как закреплять материал . . . 27

Редактор и машина Тьюринга: понимать алгоритм, выполнять его точно, находить закономерности и проверять решение на Python.

Подробный учебный конспект с разобранными примерами, практикой и ответами. Большинство примеров составлено специально для объяснения; это не копия банка заданий. Подбор тем опирается на два раздела «Решу ЕГЭ», указанных в запросе. Обозначения и конкретные правила всегда нужно сверять с условием решаемой задачи.

1. Что именно нужно понять

В задании описывается исполнитель: воображаемый механизм, который умеет выполнять небольшой набор строго определённых команд. Ему дают исходные данные и программу, то есть команды с указанием порядка их выполнения. Нужно установить результат работы или подобрать исходные данные, при которых получится нужный результат.

Исполнитель ничего не угадывает. Если в программе написано заменить одну цепочку символов, он не заменяет все похожие цепочки. Если после первой команды появилась возможность выполнить следующую, решение о её выполнении определяется устройством программы.

У обоих исполнителей удобно задавать три вопроса: «Что есть сейчас?», «Какую команду нужно выполнить сейчас?», «Что изменится после этой команды?». Затем вопросы повторяются, пока не выполнено условие остановки.

У Редактора данные - одна строка, например `111223`. Он ищет в ней соседние символы и заменяет найденный фрагмент. У машины Тьюринга данные записаны в отдельных ячейках ленты. Она читает одну ячейку, меняет её содержимое, перемещается и выбирает дальнейшие действия с учётом своего состояния.

Что сравниваем	Редактор	Машина Тьюринга
Где хранятся данные	В строке	В ячейках ленты
Что обрабатывается командой	Найденная цепочка	Одна текущая ячейка
Что задаёт дальнейшее действие	Место в программе и условия	Состояние и прочитанный символ
Как задаётся остановка	Условие цикла становится ложным	Специальная команда по условию

Начинать полезно с коротких примеров вручную. Это позволяет проверить смысл команд. Для длинных строк и перебора чисел удобно написать точное воспроизведение алгоритма на Python. После этого можно искать закономерность, которая объяснит результат и сократит вычисления.

2. Редактор: из чего состоит строка

Строка - упорядоченная последовательность символов. В строке `12012` пять символов: `1`, `2`, `0`, `1`, `2`. Порядок важен: `12` и `21` - разные строки.

Цифра в такой задаче сначала является символом. Строка `0012` содержит четыре символа и два начальных нуля. Превращать её в число 12 при выполнении замен нельзя: изменятся исходные данные.

Непрерывный фрагмент строки называется подстрокой. В строке `51273` есть подстрока `127`, потому что эти символы стоят подряд. Подстроки `13` нет: между единицей и тройкой расположены другие символы.

Говорят, что цепочка «входит» в строку, когда она встречается в ней как непрерывный фрагмент. Место такого появления называют вхождением. Одна цепочка может иметь несколько вхождений. У `11` в строке `11111` три возможных начала: на первом, втором и третьем символах. Эти вхождения перекрываются: используют некоторые общие позиции.

Команда `нашлось(v)` только отвечает на вопрос, есть ли в строке цепочка `v`. Ответ «истина» означает «да», «ложь» означает «нет». Это логические значения: два возможных результата проверки. Сама проверка строку не изменяет.

Команда `заменить(v, w)` заменяет первое слева вхождение `v` на `w`. Сначала выбирается самое левое начало подходящей цепочки; затем заменяется только этот фрагмент. Если `v` отсутствует, строка остаётся прежней. Именно так команды определены в указанном разделе банка [1].

Например, после `заменить(111, 2)` строка `11111` станет `211`: первые три единицы превратятся в двойку, последние две сохранятся. Строка `112` получилась бы при выборе другого вхождения, что нарушает правило «первое слева».

В строке `111111` та же одна команда даст `2111`. Замена второго блока единиц потребует следующего выполнения команды. Повторять замену автоматически «до конца» нельзя.

Новая цепочка может быть короче исходной, длиннее или иметь ту же длину. Если заменить `12` на `444`, длина увеличится на один. Если заменить `111` на `2`, уменьшится на два. За одну успешную замену длина меняется на «длина новой цепочки минус длина старой».

После замены нужно рассматривать всю полученную строку. Новое вхождение может появиться на границе заменённого фрагмента. Например, `11211` после замены `2` на `1` превращается в `11111`. Теперь в строке есть `111`, хотя раньше этой цепочки не было.

3. Как условия управляют командами

3.1. «И», «ИЛИ» и «НЕ»

Условия могут объединять несколько проверок. «А И В» истинно, когда истинны обе проверки. «А ИЛИ В» истинно, когда истинна хотя бы одна. В частности, обычное логическое «ИЛИ» допускает, что истинны сразу обе проверки. «НЕ А» меняет ответ проверки на противоположный.

А	В	А И В	А ИЛИ В
ложь	ложь	ложь	ложь
ложь	истина	ложь	истина
истина	ложь	ложь	истина
истина	истина	истина	истина

Поэтому цикл с условием `нашлось(111) ИЛИ нашлось(33)` продолжает работу, пока остаётся хотя бы одна из этих цепочек. Чтобы такой цикл остановился, должны отсутствовать обе. Для условия с «И» достаточно исчезновения одной цепочки.

Условие `НЕ нашлось(00)` означает, что `00` пока отсутствует. Цикл с таким условием может, наоборот, ждать появления `00`. Нельзя рассуждать по привычке, что цикл всегда «уничтожает то, что написано в условии».

В сложном условии скобки показывают, какие проверки объединены. При переводе на Python полезно сохранять смысловые скобки: это делает порядок вычисления очевидным.

3.2. «ЕСЛИ» и «ИНАЧЕ»

В конструкции «ЕСЛИ условие ТО команда1 ИНАЧЕ команда2» выбирается одна ветвь. Условие проверяется, после чего выполняется либо команда1, либо команда2. Обе ветви одной такой конструкции за один проход не выполняются.

```
ЕСЛИ нашлось(111)
    ТО заменить(111, 2)
    ИНАЧЕ заменить(2, 3)
КОНЕЦ ЕСЛИ
```

На строке `111` получится `2`. Ветка с заменой двойки в этом проходе не выполняется. Появление двойки после первой замены не запускает «ИНАЧЕ»: выбор ветви уже сделан.

Два независимых «ЕСЛИ» означают другое:

```
ЕСЛИ нашлось(111)
    ТО заменить(111, 2)
КОНЕЦ ЕСЛИ
ЕСЛИ нашлось(2)
    ТО заменить(2, 3)
КОНЕЦ ЕСЛИ
```

На строке `111` первая проверка разрешит замену на `2`. Затем вторая проверка увидит уже новую строку и разрешит замену на `3`. Итогом станет `3`.

В цепочке «ЕСЛИ ... ИНАЧЕ ЕСЛИ ... ИНАЧЕ ...» проверки идут по порядку до первой подходящей ветви. Это задаёт приоритет правил. В строке `22111` правило для `111` может быть выбрано раньше правила для `22`, если именно так устроена программа. Правило «первое слева» определяет вхождение внутри уже выбранной команды, а не выбор между разными командами.

3.3. «ПОКА» и один проход цикла

Цикл - это повторение группы команд. В конструкции «ПОКА условие ... КОНЕЦ ПОКА» условие проверяется перед каждым выполнением этой группы. Команды внутри называют телом цикла. Одно полное выполнение тела называют проходом, или итерацией.

Если условие ложно уже в начале, тело не выполнится ни разу. Если условие истинно, команды тела исполняются по очереди. Затем исполнитель возвращается к проверке условия.

```
ПОКА нашлось(12)
    заменить(12, 3)
    заменить(3, 4)
КОНЕЦ ПОКА
```

На строке `12` первый проход даст `12 → 3 → 4`. Хотя после первой замены условие `нашлось(12)` уже стало ложным, вторую команду всё равно нужно выполнить. Повторная проверка стоит только перед следующим проходом.

Если в теле три независимых «ЕСЛИ», за проход возможны от нуля до трёх успешных замен. Если в теле один выбор «ЕСЛИ ... ИНАЧЕ», выполняется одна ветвь, но внутри этой ветви тоже может находиться несколько команд. Число проходов, выполненных команд и успешных замен нужно различать.

4. Полный пример Редактора вручную

Рассмотрим программу:

```
ПОКА нашлось(111) ИЛИ нашлось(33)
    ЕСЛИ нашлось(33)
        ТО заменить(33, 1)
        ИНАЧЕ заменить(111, 3)
    КОНЕЦ ЕСЛИ
КОНЕЦ ПОКА
```

Пусть на вход подано восемь единиц. Сначала записываем исходную строку целиком. Перед каждым проходом проверяем условие. Затем определяем ветвь и выполняем одну замену.

Проход	Строка перед проходом	Что выбирается	Строка после прохода
1	11111111	33 нет, заменить 111 на 3	311111
2	311111	33 нет, заменить 111 на 3	3311
3	3311	33 есть, заменить 33 на 1	111
4	111	33 нет, заменить 111 на 3	3

Следующая проверка ложна: в `3` нет ни `111`, ни `33`. Программа остановилась. Ответ - строка `3`.

Обрати внимание на третий проход: после замены 33 на 1 оставшиеся справа две единицы присоединяются к новой единице. Появляется 111. Это результат изменения всей строки и её границы.

Удобная таблица ручного выполнения содержит текущую строку, истинность нужной проверки, выбранную команду и результат. Старайся не выполнять несколько преобразований в уме, пока смысл команд не стал привычным.

Если исходная строка известна только по количеству одинаковых символов, можно сначала взять 6-10 символов. Такой пример помогает увидеть механизм, но его результат ещё не доказывает правило для всех длин.

5. Как точно воспроизвести Редактор на Python

5.1. Минимум языка, который здесь нужен

В Python запись `'111'` - строка из трёх символов. Кавычки обозначают границы строки и не входят в её содержимое. Запись `s = '111'` связывает имя `s` с этой строкой. Здесь `=` означает присваивание: «сохрани значение справа под именем слева». Для проверки равенства используется `==`.

Сложение строк склеивает их: `'1' + '22'` даст `'122'`. Умножение строки на целое неотрицательное число повторяет её: `'8' * 4` даст `'8888'`. Это не арифметическое умножение цифры восемь.

Поэтому строка из одной единицы и 80 восьмёрок записывается как `'1' + '8' * 80`. Её длина равна 81. Если сказано «длина 80, первая цифра 1, остальные 8», восьмёрок должно быть 79.

Проверка `'111' in s` соответствует `нашлось(111)`. Проверка `'00' not in s` означает отсутствие `00`. Слова `and`, `or`, `not` соответствуют «И», «ИЛИ», «НЕ».

В условии	На Python
`нашлось(111)`	`'111' in s`
`НЕ нашлось(00)`	`'00' not in s`
`нашлось(111) ИЛИ нашлось(33)`	`'111' in s or '33' in s`
`заменить(111, 3)`	`s = s.replace('111', '3', 1)`
`ЕСЛИ ... ИНАЧЕ`	`if ...: ... else: ...`
`ПОКА ...`	`while ...:`

У `replace` три аргумента: что заменить, на что заменить и сколько первых вхождений заменить. Для Редактора третий аргумент должен быть `1`. Без него Python заменяет все найденные неперекрывающиеся вхождения за этот вызов. Это поведение описано в официальной документации [3].

`replace` возвращает новую строку, поэтому результат сохраняют: `s = s.replace(...)`. Если написать только `s.replace(...)`, значение переменной `s` не обновится.

```
s = '111111'
print(s.replace('111', '2', 1))  # 2111
print(s.replace('111', '2'))     # 22
```

В обоих вызовах выше используется прежняя `s`, потому что результат только печатается. Чтобы продолжать преобразования с изменённой строкой, нужно присваивание.

5.2. Перевод разобранной программы

```
s = '1' * 8

while '111' in s or '33' in s:
    if '33' in s:
        s = s.replace('33', '1', 1)
    else:
        s = s.replace('111', '3', 1)

print(s)  # 3
```

После заголовков `while`, `if`, `else` ставится двоеточие. Отступы показывают, какие команды входят в соответствующую группу. Обычно используют четыре пробела на каждый уровень. Здесь `print` без отступа относится к командам после цикла: он выводит итог один раз.

Если в условии два независимых «ЕСЛИ», в Python нужно два `if`. Если второе является «ИНАЧЕ ЕСЛИ», используется `elif`. Замена `if` на `elif` меняет поведение программы, поэтому переводить нужно структуру, а не только список замен.

Плохая запись `while '111' or '33' in s:` не означает поиск двух цепочек. Непустая строка `'111'` сама рассматривается Python как истинное значение. Правильно: `while '111' in s or '33' in s:`. У каждой проверяемой цепочки должно быть своё `in s`.

Для проверки первых шагов можно печатать строку после каждого прохода:

```
s = '1' * 8
step = 0

while '111' in s or '33' in s:
    if '33' in s:
        s = s.replace('33', '1', 1)
    else:
        s = s.replace('111', '3', 1)
    step += 1
    print(step, s)
```

`step` - счётчик. Запись `step += 1` означает увеличить его на один. Такой вывод позволяет сопоставить программу с таблицей ручного выполнения.

5.3. Что именно выводить

Для `s = '1114'` сама строка равна 1114, длина равна 4, количество единиц равно 3, сумма цифр равна 7. Это четыре разных ответа.

```
print(s)             # строка
print(len(s))        # количество символов
print(s.count('1'))   # количество единиц

total = 0
for digit in s:
    total += int(digit)
print(total)         # сумма цифр
```

В цикле `for digit in s` переменная `digit` по очереди получает каждый символ. `int(digit)` превращает один символ-цифру в соответствующее число, чтобы его можно было прибавить. Сокращённая запись той же суммы - `sum(int(digit) for digit in s)`.

Если в строке остался служебный знак, например >, напрямую превращать его в число нельзя. Нужно либо удалить этот знак для подсчёта, либо заранее считать только известные цифры. Если в строке есть только 2, 3 и >, сумма равна `2 * s.count('2') + 3 * s.count('3')`.

Для одной цифры `count` считает её количество обычным образом. Для цепочки из нескольких символов Python считает неперекрывающиеся вхождения: `'111'.count('11')` равно 1. Это не число всех возможных начал цепочки `11`.

6. От отдельных замен к закономерности

6.1. Повторение одинакового преобразования

Вернёмся к программе `111 → 3`, `33 → 1` с приоритетом `33`. Если исходная строка состоит из `n` единиц и `n ≥ 6`, первые три прохода выглядят так:

```
n единиц
→ одна тройка и n - 3 единицы
→ две тройки и n - 6 единиц
→ n - 5 единиц
```

После этих трёх проходов снова получена строка только из единиц. Программа находится перед той же проверкой цикла. Ситуация повторяется полностью, но единиц стало на пять меньше. Это и позволяет повторять рассуждение.

Здесь существенны обе части: сохранился вид строки и уменьшился параметр. Просто заметить, что «за несколько замен уходит пять символов», недостаточно, если новая строка устроена иначе и следующие замены пойдут по-другому.

Для 30 единиц получаем количества `30 → 25 → 20 → 15 → 10 → 5`. На пяти единицах уже нельзя выполнить описанную группу из трёх проходов: для неё нужно хотя бы шесть. Остаток `11111` нужно обработать напрямую: он становится `311` и останавливается.

Ответ для 30 единиц - `311`. Пустая строка не получается. Именно поэтому механическое использование остатка от деления `30 % 5 = 0` дало бы ошибку.

Достаточно отдельно разобрать пять малых исходных длин:

Единиц в малой строке	Итог
1	1
2	11
3	3
4	31
5	311

Для положительного `n` приведение к этим случаям выражается так: `r = (n - 1) % 5 + 1`. Значение `r` всегда лежит от 1 до 5. Но формула полезна только после доказательства повторяющегося преобразования и его границы `n ≥ 6`.

6.2. Почему программа завершится

В нашей программе `111 → 3` уменьшает длину на два, а `33 → 1` - на один. Если условие цикла истинно, одна из замен обязательно возможна и выполняется. Значит, каждый проход строго уменьшает длину. Длина - целое неотрицательное число, бесконечно уменьшаться она не может. Следовательно, программа завершится.

Не всякая программа Редактора сокращает строку. При замене `1 → 11` она растёт. Это само по себе ещё не доказывает бесконечность: иногда растёт обработанная часть, а необработанная сокращается.

Например, пусть исходная строка состоит из одной двойки и `n` троек:

```
ПОКА нашлось(2)
    ЕСЛИ нашлось(23)
        ТО заменить(23, 3332)
        ИНАЧЕ заменить(2, 333)
    КОНЕЦ ЕСЛИ
КОНЕЦ ПОКА
```

Двойка здесь отмечает границу. При `23 → 3332` одна ещё не обработанная тройка справа от двойки превращается в три тройки слева. Число троек справа уменьшается на один. Созданные слева тройки уже не обрабатываются: двойка движется от них вправо.

После `n` таких замен справа от двойки ничего не останется. Следующий проход заменит саму двойку на три тройки. Итог содержит `3n + 3` троек, сумма цифр равна `9n + 9`, число проходов равно `n + 1`. При `n = 4` получаем 15 троек, сумму 45 и 5 проходов.

Для доказательства остановки здесь считаем необработанные символы, а не общую длину.

6.3. Что остаётся неизменным

Иногда замены сохраняют некоторую величину. Такую величину называют инвариантом. Например, замена `12 → 3` сохраняет сумму цифр: `1 + 2 = 3`. Другой пример - сохранение не самой суммы, а её остатка от деления на число.

У программы `111 → 3`, `33 → 1` первая замена сохраняет сумму цифр, а вторая уменьшает её на пять. Значит, остаток суммы при делении на пять не меняется. Для 30 исходных

единиц итоговая сумма должна делиться на пять. У строки `311` сумма равна пяти, что согласуется с этим выводом.

Один инвариант обычно не определяет ответ полностью. Разные строки могут иметь одну сумму или одинаковый остаток. Инвариант помогает отбросить невозможные варианты и проверить найденный ответ. Чтобы объявить вариант ответом, нужно учитывать порядок замен и достижимость результата.

7. Подсчёт замен, неизвестный порядок и маркеры

7.1. Баланс количества символов

Если нас спрашивают о количествах символов, иногда можно не выписывать всю строку. Обозначим через `a` число успешных замен `111 → 3`, а через `b` число успешных замен `33 → 1`. В начале имеется `n` единиц и ни одной тройки.

Первая замена убирает три единицы и добавляет одну тройку. Вторая убирает две тройки и добавляет одну единицу. Поэтому после завершения:

```
количество единиц = n - 3a + b
количество троек = a - 2b
длина = n - 2a - b
сумма цифр = n - 5b
```

Для уже найденного результата `311` при `n = 30` сумма равна 5. Значит, `30 - 5b = 5`, откуда `b = 5`. В итоге одна тройка: `a - 2b = 1`, поэтому `a = 11`. Всего 16 успешных замен. Длина проверяется: `30 - 2 * 11 - 5 = 3`.

Эти равенства описывают изменения количеств, но не доказывают существование нужного порядка команд. Числа `a` и `b` должны быть целыми неотрицательными и соответствовать реальному выполнению программы. В этом примере достижимость уже установлена отдельным разбором.

7.2. Почему порядок исходных цифр нельзя угадывать

Если условие задаёт только количество цифр и говорит, что они расположены произвольно, это не даёт права сложить одинаковые цифры в блоки. Для программы `ПОКА нашлось(12): заменить(12, 4)` строки `12` и `21` имеют одинаковые количества единиц и двоек. Однако первая превратится в `4`, а вторая останется `21`.

Случайное перемешивание может помочь заметить гипотезу, но не доказывает минимум или максимум: нужный порядок может ни разу не попасться. Перебор одного произвольного расположения также ничего не доказывает об остальных.

Рассмотрим задачу: в исходной строке 12 единиц и неизвестное число `b` двоек; выполняется `12 → 4`, пока возможно; итоговая сумма цифр равна 30. Нужно минимальное `b`.

Исходная сумма равна `12 + 2b`. Каждая успешная замена увеличивает её на один, потому что `4 - (1 + 2) = 1`. Если замен было `k`, то `30 = 12 + 2b + k`. Каждая замена расходует одну единицу и одну двойку, поэтому `0 ≤ k ≤ min(12, b)`.

Отсюда итоговая сумма не превосходит $12 + 3b$. Чтобы получить 30, необходимо $b \geq 6$. Но одной необходимой границы недостаточно: нужно показать, что шести двоек действительно хватит.

Берём шесть блоков 12, после них шесть единиц: всего 12 единиц и 6 двоек. Выполняются ровно шесть замен. Получается шесть четвёрок и шесть единиц с суммой $6 * 4 + 6 = 30$. Граница достижима, поэтому минимум равен 6.

Хорошее решение задачи на минимум состоит из двух частей: меньше быть не может; указанное значение можно получить. Для максимума рассуждение устроено аналогично.

7.3. Служебный символ как граница обработки

В некоторых программах появляется символ, который помогает отделить обработанную часть строки от необработанной. Его удобно называть маркером. Это обычный символ данных со специально организованными правилами замены, а не особая встроенная возможность Редактора.

Пусть в начале стоит один знак >, за ним цифры 1, 2 и 3 в любом порядке. Правила выполняются в таком порядке:

```
ПОКА нашлось(>1) ИЛИ нашлось(>2) ИЛИ нашлось(>3)
    заменить(>1, 22>)
    заменить(>2, 3>)
    заменить(>3, 5>)
КОНЕЦ ПОКА
```

Каждая успешная замена обрабатывает ровно одну исходную цифру справа от маркера и кладёт результат слева от него. Из 1 получаются две двойки, из 2 - тройка, из 3 - пятёрка. Маркер сохраняется в одном экземпляре и движется по исходным цифрам вправо.

Заметим, что все три команды стоят в теле независимо. Если после обработки единицы сразу идёт двойка, в том же проходе сможет сработать и вторая команда. Количество проходов может быть меньше количества обработанных цифр.

Если исходных единиц a, двоек b, троек c, итоговая сумма равна $4a + 3b + 5c$. Цифр в результате будет $2a + b + c$; общая длина с оставшимся маркером равна $2a + b + c + 1$. Эти величины не зависят от первоначального порядка цифр, потому что каждый исходный символ обрабатывается ровно один раз по своему правилу.

Например, >111223 превращается в 222222335>. Сумма равна $4 * 3 + 3 * 2 + 5 = 23$. Маркер не является цифрой и не участвует в этой сумме. Порядок цифр результата в общем случае зависит от порядка входа; независимость суммы не означает независимость всей строки.

8. Перебор неизвестного параметра

Перебор означает последовательно проверить допустимые варианты. Для каждого варианта создаётся исходная строка, программа выполняется до остановки, затем проверяется нужное свойство результата.

8.1. Минимум и точная граница

Пусть в программе из главы 4 исходная строка состоит из `n` единиц, `102 < n ≤ 200`. Нужно минимальное `n`, при котором итог равен `1`.

```
for n in range(103, 201):
    s = '1' * n

    while '111' in s or '33' in s:
        if '33' in s:
            s = s.replace('33', '1', 1)
        else:
            s = s.replace('111', '3', 1)

    if s == '1':
        print(n)  # 106
        break
```

`range(103, 201)` даёт целые числа от 103 до 200 включительно: правая граница в `range` не входит. Начало 103 соответствует строгому условию «больше 102». Начальная строка создаётся заново внутри внешнего цикла.

Проверка `s == '1'` стоит после внутреннего `while`, но внутри `for`. Команда `break` завершает ближайший охватывающий цикл, здесь внешний `for`, потому что внутренний цикл уже закончился. При проверке кандидатов по возрастанию первый подходящий и есть минимальный.

Если верхней границы в условии нет, нельзя автоматически утверждать, что её роль играет 1000 или 10000. Для минимума последовательная проверка с нижней границы обоснует ответ, когда найден первый подходящий кандидат и все предыдущие проверены. Для доказательства отсутствия ответа или конечного максимума нужна обоснованная область поиска.

В нашем примере закономерность даёт дополнительную проверку: результат `1` получается при `n = 1, 6, 11, ...`. Первое такое число, большее 102, равно 106.

8.2. Что сохранять при поиске максимума

Максимальный результат и значение `n`, на котором он получен, - разные величины. Если вопрос просит `n`, сохранять нужно `n`. Если нужно значение суммы, сохранять нужно сумму. При одинаковом максимуме несколько значений `n` могут подходить; условие может просить наименьшее или наибольшее из них.

```
best_total = -1
best_n = None

for n in range(1, 101):
    s = '1' * n
    while '111' in s or '33' in s:
        if '33' in s:
            s = s.replace('33', '1', 1)
        else:
            s = s.replace('111', '3', 1)

    total = sum(int(digit) for digit in s)
    if total > best_total:
        best_total = total
        best_n = n

print(best_total, best_n)  # 5 5
```

Здесь `None` обозначает, что подходящий параметр ещё не сохранён. Благодаря порядку возрастания и строгому `>` остаётся наименьшее `n` среди вариантов с максимальной суммой. Если заменить `>` на `>=`, при равенстве прежний кандидат будет заменяться новым и сохранится наибольшее `n`.

Число подходящих входов и число различных выходных строк тоже отличаются: разные входы могут дать один результат. Для хранения только различных строк в Python используют множество: `results = set()`, затем `results.add(s)`, в конце `len(results)`.

8.3. Условие «сумма является простым числом»

Простое число - целое число больше 1, у которого только два положительных делителя: 1 и оно само. Поэтому 0 и 1 не являются простыми. Чтобы проверить число `x`, достаточно искать делитель от 2, пока `d * d ≤ x`. Если составное число равно произведению двух множителей, хотя бы один из них не больше квадратного корня: иначе их произведение было бы больше `x`.

```
def is_prime(x):
    if x < 2:
        return False
    d = 2
    while d * d <= x:
        if x % d == 0:
            return False
        d += 1
    return True
```

`def` создаёт функцию: набор действий с именем, который можно вызвать для разных значений `x`. `return` возвращает результат и завершает текущий вызов функции. `%` вычисляет остаток от деления. Поэтому `x % d == 0` означает, что `x` делится на `d` без остатка.

У маркерного примера при трёх единицах, `n` двойках и одной тройке сумма равна `17 + 3n`. Для `10 < n ≤ 100` минимальное `n`, дающее простую сумму, можно искать уже без строк: для 11 сумма 50, для 12 сумма 53. Ответ - 12. Сокращение поиска законно, потому что формула для суммы была предварительно выведена из работы маркера.

9. Сколько различных конечных строк возможно

Остановка иногда позволяет описать форму результата, даже не выполняя все замены. Например, если цикл имеет условие `нашлось(111) ИЛИ нашлось(22)` и завершился, в конечной строке отсутствуют и `111`, и `22`.

Пусть исходная строка может быть любой строкой из единиц и двоек без дополнительных ограничений. Сколько различных конечных строк с тремя двойками может быть у завершающейся программы с таким условием?

Между соседними двойками должна быть хотя бы одна единица, иначе появится `22`. Больше двух единиц подряд быть не может, иначе появится `111`. Значит, в каждом из двух внутренних промежутков есть выбор: одна единица или две. Перед первой двойкой и после последней допустимы ноль, одна или две единицы.

Итоговое количество равно `3 * 2 * 2 * 3 = 36`. Мы перемножаем числа независимых выборов: каждому набору длин промежутков соответствует одна строка.

Здесь нужно отдельно проверить, что все посчитанные строки могут быть результатами программы. Это верно из-за разрешения подать любую строку из единиц и двоек: каждую посчитанную строку можно сразу подать на вход, и цикл выполнится ноль раз. Если исходная строка обязана состоять только из единиц или иметь заданную длину, такое обоснование уже не подходит.

Для `r ≥ 1` двоек аналогичное число строк равно `9 * 2^(r - 1)`: у двух краёв по три варианта, у `r - 1` внутренних промежутков по два. Если программа использует условие с «И» вместо «ИЛИ», ограничения на конечную строку будут другими.

10. Машина Тьюринга: лента, головка, состояние

10.1. Что находится на ленте

Представь бесконечный ряд отдельных ячеек. В каждой записан ровно один символ. Набор допустимых символов называют алфавитом. Например, алфавит может содержать `0`, `1` и специальный пустой символ `λ`.

«Пустой» символ тоже является символом в ячейке. Он означает отсутствие обычных данных. Это не цифра 0. Программа может по-разному действовать на 0 и на `λ`. При записи `λ` прежнее содержимое ячейки стирается; сама ячейка остаётся на месте, и соседние ячейки не сдвигаются.

Бесконечность ленты означает, что за конечным записанным фрагментом всегда есть ещё ячейки с пустым символом. Не нужно рисовать бесконечно много клеток: показывают данные и небольшой запас пустых клеток по краям.

Головка - устройство, которое в каждый момент читает одну текущую ячейку. Она может записать в неё символ и передвинуться. Положение головки обязательно нужно учитывать: одинаковая лента с головкой в разных местах обычно даст разную дальнейшую работу.

Для удобства ячейкам можно приписать целые координаты: `..., -2, -1, 0, 1, 2, ...`. Это наши метки для решения. Движение вправо увеличивает координату на один, влево уменьшает на один. Переходить через ноль разрешено.

10.2. Зачем нужно состояние

Один и тот же символ может требовать разных действий на разных этапах работы. Например, при первом проходе нужно дойти вправо до конца числа, а на обратном пути менять нули на единицы. Чтобы отличать эти ситуации, исполнитель хранит состояние - одну метку из заданного списка, например `q0`, `q1`, `q2`.

Удобно понимать состояние как текущий режим работы. Метка `q1` не обязана означать «шаг номер 1» или «ячейка номер 1». Машина может много шагов подряд оставаться в `q1`, а позже возвращаться в уже встречавшееся состояние.

Для полного описания текущей ситуации нужны содержимое ленты, положение головки и состояние. Иногда такое полное описание называют конфигурацией. Не путай его с одной меткой `q`: состояние `q` - лишь часть конфигурации.

10.3. Как читать таблицу команд

В задачах по указанной ссылке строки таблицы задают состояния, столбцы - читаемые символы. В нужной клетке стоит тройка «что записать, куда двигаться, в какое состояние перейти» [2]. Если в другой задаче таблицу развернули или изменили обозначения, руководствуйся её описанием.

Команда `0, L, q3` означает: в текущую ячейку записать `0`, затем переместиться на одну ячейку влево и продолжить в состоянии `q3`. Записывается символ в старой текущей ячейке, а не в той, куда головка придёт.

Обозначение	Действие в используемом здесь формате
`L`	Сдвиг на одну ячейку влево
`R`	Сдвиг на одну ячейку вправо
`N`	Остаться в той же ячейке и продолжить
`S`	Завершить работу после записи символа

Не нужно выводить значения букв из привычек других языков. Здесь `S` является остановкой, а `N` отсутствием сдвига. В другом формате остановку могут обозначать отдельным состоянием. Приоритет имеет определение в условии.

Команда `1, N, q2` запишет единицу и оставит головку на месте. На следующем шаге машина снова прочитает эту же ячейку, но уже с новым символом и в состоянии `q2`. Команда `1, S, q2` сначала запишет единицу, а затем завершит работу. Считать её «просто остановкой без изменения ленты» нельзя.

Один такт - выполнение одной выбранной команды таблицы. Сначала команду выбирают по старым состоянию и символу. Затем выполняют запись, движение и смену состояния. Новая команда выбирается только на следующем такте. Нельзя за один такт сразу выполнить несколько клеток таблицы.

Пустая клетка в таблице команд и пустая ячейка ленты - разные вещи. Пустая клетка таблицы часто помечает невозможное сочетание. Это не автоматическая команда остановки. Если при проверке ты попал в неё, нужно сверить исходное положение, состояние и предыдущие шаги.

11. Первый полный разбор машины Тьюринга

Дана следующая таблица. В начале головка находится в ближайшей пустой ячейке справа от двоичной строки, состояние `q0`.

Состояние	λ	0	1
q0	λ, L, q1	-	-
q1	λ, S, q1	1, L, q1	0, S, q1

Символ «-» в этой учебной таблице означает, что команда не задана. В `q0` головка изначально видит пустую ячейку, поэтому другие клетки этой строки не понадобятся.

Исходные данные - `10100`. Присвоим пяти ячейкам с цифрами координаты 0-4. Головка начинает на координате 5, где находится λ.

Такт	До: состояние, позиция, символ	Команда	Цифры после	Позиция после
1	q0, 5, λ	λ, L, q1	10100	4
2	q1, 4, 0	1, L, q1	10101	3
3	q1, 3, 0	1, L, q1	10111	2
4	q1, 2, 1	0, S, q1	10011	2, остановка

Остальные ячейки по-прежнему пусты. Ответ - `10011`, если спрашивают строку, или 19, если спрашивают десятичное значение двоичной записи. Тактов было четыре. Остановочная команда считается, потому что она выполнялась и изменила цифру.

В результате поменялись два нуля справа на единицы, а встретившаяся единица - на ноль. Часть левее этой единицы не изменилась. Чтение справа налево не переворачивает строку: мы продолжаем записывать содержимое ленты слева направо.

Начальное положение нужно брать буквально. «В ближайшей пустой ячейке справа» и «на крайнем правом символе» - разные позиции. В этой таблице запуск в `q0` на последней цифре сразу привёл бы к клетке без команды.

12. Как увидеть смысл всей машины

12.1. Хвост из нулей и вычитание единицы

У таблицы из главы 11 состояние `q0` только вводит головку в строку. В `q1` она идёт влево по конечным нулям, заменяя их единицами. На первой единице записывает ноль и останавливается.

Для любой строки, содержащей единицу, это можно записать так: `P 1 0^t → P 0 1^t`. Здесь `P` обозначает всю неизменяемую левую часть, а `0^t` в этой записи означает `t` одинаковых нулей подряд. Это сокращение для строки, а не число ноль, возведённое в степень. Аналогично `1^t` - `t` единиц.

Так работает вычитание единицы в двоичной записи. В `10100` справа стоят два нуля, поэтому получаем `10011`. Если справа сразу единица, например `10101`, она меняется на ноль: получается `10100`.

Если строка состоит только из нулей, единица не встретится: машина заменит все нули единицами, дойдёт до `λ` слева и остановится. Поэтому для `000` результат `111`, а не запись отрицательного числа. Интерпретация «уменьшить число на один» требует оговорки о ненулевом исходном числе.

У ненулевой строки с `t` нулями в конце число тактов равно `t + 2`: один для входа из пустой ячейки, `t` для нулей, один для остановочной замены единицы. У строки из `L` нулей будет `L + 2` такта: вход, обработка всех нулей, остановка на пустой ячейке.

12.2. Минимум и максимум исходных нулей

Пусть длина строки равна `L`, а после работы этой машины в ней `K` нулей, причём `1 ≤ K ≤ L`. Нужно выяснить, сколько нулей могло быть в начале.

Нули результата состоят из нулей неизменённого фрагмента `P` и одного нового нуля, полученного из единицы. Поэтому в `P` было `K - 1` нулей. Если нулевой хвост имел длину `t`, то исходных нулей было `K - 1 + t`.

Для минимума берём `t = 0`: единица стоит сразу в конце и превращается в новый ноль. Минимум равен `K - 1`.

Для максимума исходная строка может содержать ровно одну единицу и `L - 1` нулей. Нужно поставить `K - 1` нулей слева от неё, а остальные `L - K` - справа. Тогда после преобразования будет именно `K` нулей. Больше `L - 1` исходных нулей невозможно: полностью нулевая строка дала бы ноль нулей в результате, а у нас `K ≥ 1`.

При `L = 12`, `K = 5` максимум равен 11. Он достигается на строке из четырёх нулей, единицы и семи нулей; результат содержит пять нулей и семь единиц. Минимум равен 4: можно взять семь единиц, четыре нуля и последнюю единицу. Последняя единица превратится в пятый ноль.

Особый случай `K = 0` рассматривается отдельно: для данной таблицы результат без нулей получается только из полностью нулевого входа. Поэтому исходных нулей тогда ровно `L`. Нельзя подставлять `K = 0` в формулу для минимума и получать -1.

12.3. Независимая обработка каждой цифры

Бывает машина, которая проходит всю строку справа налево в одном состоянии, заменяет `5` и `7` на `1`, а `9` на `0` и останавливается на пустой ячейке. В таком случае каждый исходный символ обрабатывается один раз, соседние позиции не меняются местами.

Если итоговая строка из нулей и единиц должна иметь разные соседние символы, она обязана чередоваться: `1010...` либо `0101...`. Для девяти символов в одном варианте пять единиц и четыре нуля, в другом наоборот.

Каждая итоговая единица могла возникнуть из 5 или 7. Для минимальной суммы выбираем 5. Итоговый ноль обязательно возник из 9. Значит, выгоднее иметь больше единиц: `5 * 5 + 4 * 9 = 61`. Достигающий вход - `595959595`, итог - `101010101`.

Здесь порядок важен из-за чередования, но перебирать все строки из трёх цифр не нужно. Таблица превращается в соответствие исходных и конечных символов, затем учитывается условие на результат.

12.4. Состояния могут хранить чётность

Состояние иногда хранит одно небольшое свойство уже просмотренной части. Например, `q_even` может означать, что просмотрено чётное количество единиц, а `q_odd` - нечётное. При чтении нуля состояние не меняется; при чтении единицы переключается на другое.

Если головка начинает на первой цифре в `q_even`, а машина идёт вправо, после просмотра всего слова состояние показывает чётность числа единиц. На пустой ячейке можно записать `0` в чётном состоянии и `1` в нечётном и остановиться. Для `1011` единиц три, поэтому будет дописана единица: `10111`.

Это объяснение становится доказательством, когда проверены обе строки таблицы: каждый ноль сохраняет правильный смысл состояния, а каждая единица меняет чётность и переводит в нужную строку. Названия состояний сами по себе ничего не гарантируют: смысл устанавливается по командам.

13. Двоичные числа и несколько проходов по ленте

13.1. Дописывание справа

В двоичной системе каждый следующий разряд справа имеет вдвое меньший вес, чем предыдущий. Если к записи числа `N` справа дописать ноль, все прежние цифры сдвинутся на один разряд влево, поэтому число станет `2N`. Дописывание единицы даёт `2N + 1`.

Например, `37 = 100101` в двоичной записи. После дописывания единицы получится `1001011`, то есть `2 * 37 + 1 = 75`. Если таблица просто проходит цифры вправо без изменений, а на первом `λ` пишет `1` и останавливается, именно это и происходит.

При дописывании `k` битов справа исходное число умножается на `2^k`, затем прибавляется значение дописанного блока. Приписывание единицы слева к записи длины `k` добавляет `2^k`: положение нового разряда зависит от длины прежней записи.

```
print(bin(37))           # 0b100101
print(bin(37)[2:])       # 100101
print(int('1001011', 2)) # 75
```

`bin` строит двоичную запись с префиксом `0b`. Запись `[2:]` берёт символы начиная с позиции 2 до конца, убирая префикс; позиции в Python считаются с нуля. `int(строка, 2)` читает строку как двоичную запись. Эти операции используют для перевода систем счисления, а не для хранения строки с заданными начальными нулями.

13.2. Как читать машину по этапам

В сложной таблице сначала проследи, где машина меняет состояние. Иногда вся работа естественно делится на этапы: войти в данные, пройти вправо, вернуться влево, дописать начало, остановиться. Для каждого этапа выясни направление, изменение цифр и событие перехода к следующему состоянию.

Рассмотрим таблицу. Вход - непустая двоичная строка, головка начинает на ближайшем `λ` слева в состоянии `q0`.

Состояние	λ	0	1
q0	λ, R, q1	-	-
q1	0, L, q2	1, R, q1	1, R, q1
q2	0, L, q3	-	1, L, q2
q3	1, S, q3	-	-

В `q1` все исходные цифры превращаются в единицы, головка идёт вправо. На правой пустой ячейке она пишет ноль и возвращается влево в `q2`. В `q2` проходит полученные единицы без изменений. На пустой ячейке слева пишет ноль, сдвигается ещё левее и переходит в `q3`. Там пишет единицу и останавливается.

Если исходная длина равна `k`, итог имеет вид `10`, затем `k` единиц, затем `0`. Порядок выполнения команд слева - сначала записать 0, затем ещё левее 1 - даёт префикс `10` при чтении ленты слева направо.

k	Итоговая двоичная строка	Значение
1	1010	10
2	10110	22
3	101110	46
4	1011110	94
5	10111110	190
6	101111110	382
7	1011111110	766

Если спрашивают максимальный возможный результат не больше 500, ответ 382. При `k = 7` уже получается 766, и дальше значение только растёт. Перебирать все исходные числа не нужно: их цифры в первом проходе всё равно становятся единицами, влияет только длина.

Формула этого результата: `R = 3 * 2^(k + 1) - 2`. Она получается из значения префикса `10`, равного 2: после приписывания `k` единиц будет `2 * 2^k + (2^k - 1)`, затем конечный ноль удваивает число. Можно пользоваться и таблицей: формула не обязательна, если смысл этапов понятен.

14. Универсальное воспроизведение машины на Python

14.1. Почему для ленты удобен словарь

Обычный список подходит, если заранее известно, сколько свободных ячеек нужно слева и справа. Но в Python индекс `-1` у списка означает последний элемент, а не новую клетку слева. Поэтому прямой запуск с отрицательными координатами легко даёт ошибку.

Словарь позволяет хранить пары «координата - символ». Например, `{0: '1', 1: '0'}` означает единицу в клетке 0 и ноль в клетке 1. Если координата не записана в словаре, считаем ячейку пустой.

Команда `tape.get(pos, 'λ')` читает символ по координате `pos`; если такой координаты нет, возвращает `λ`. Команда `tape[pos] = symbol` записывает символ в эту клетку. Отрицательные координаты работают так же, как положительные.

Правила тоже удобно хранить в словаре. Его ключом будет пара `(state, symbol)`, а значением тройка `(что_записать, движение, новое_состояние)`. Пара или тройка в круглых скобках - несколько значений, объединённых в одну запись; такую запись Python называет кортежем.

14.2. Правила для машины из главы 11

```
rules = {
    ('q0', 'λ'): ('λ', 'L', 'q1'),
    ('q1', 'λ'): ('λ', 'S', 'q1'),
    ('q1', '0'): ('1', 'L', 'q1'),
    ('q1', '1'): ('0', 'S', 'q1'),
}
```

14.3. Само выполнение

```
def run_tm(word, rules, start_pos, start_state='q0',
           max_steps=100000):
    tape = {}
    for i, symbol in enumerate(word):
        tape[i] = symbol

    pos = start_pos
    state = start_state
    steps = 0

    while steps < max_steps:
        symbol = tape.get(pos, 'λ')
        key = (state, symbol)
        if key not in rules:
            raise RuntimeError('Нет команды для ' + str(key))

        write, move, next_state = rules[key]
        tape[pos] = write
        state = next_state
        steps += 1

        if move == 'S':
            return tape, pos, state, steps
        elif move == 'L':
            pos -= 1
        elif move == 'R':
            pos += 1
        elif move != 'N':
            raise RuntimeError('Неизвестное движение')

    raise RuntimeError('Достигнут предел числа шагов')
```

`enumerate(word)` выдаёт по порядку номер символа и сам символ. Поэтому первые четыре строки функции записывают слово в клетки от 0 до `len(word) - 1`. Строка `write, move, next_state = rules[key]` распределяет три части найденной команды по трём переменным.

Каждый проход `while` выполняет ровно один такт. Сначала читаются прежний символ и прежнее состояние и выбирается команда. Затем выполняется запись. При `S` функция возвращает результат уже после записи. При `N` координата остаётся прежней, но выполнение продолжается.

`raise RuntimeError(...)` сообщает об ошибке и прекращает запуск с пояснением. Отсутствующее правило не считается штатной остановкой. Ограничение `max_steps` защищает от слишком долгого запуска; его достижение не означает, что найден ответ или доказан цикл.

```
word = '10100'
tape, pos, state, steps = run_tm(
    word, rules, start_pos=len(word)
)

result = ''
for i in range(len(word)):
    result += tape.get(i, 'λ')

print(result)             # 10011
print(pos, state, steps)  # 2 q1 4
```

Здесь можно читать только прежние пять позиций, потому что именно эта машина не добавляет цифры за их границами. Для машины, которая дописывает цифры, нужно прочитать весь изменённый фрагмент ленты.

```
left = min(tape)
right = max(tape)
fragment = ''
for i in range(left, right + 1):
    fragment += tape.get(i, 'λ')
print(fragment.strip('λ'))
```

`min(tape)` и `max(tape)` дают минимальную и максимальную записанную координату. `strip('λ')` убирает пустые символы только с двух краёв. Если внутри есть `λ`, его нельзя автоматически удалить: это отдельная клетка между данными. Для условий, где нужен один сплошной двоичный блок, сначала убедись, что результат действительно имеет такой вид.

14.4. Как проверять гипотезы программой

Сопоставь короткий запуск с ручной таблицей, затем проверь характерные случаи. Для машины вычитания это, например, `1`, `0`, `000`, `10101`, `10100`, `1000`. Такие варианты проверяют остановку на единице, остановку на пустой ячейке, нулевой хвост разной длины и сохранение начальных нулей.

У Редактора можно сохранять все строки, которые были перед проверкой цикла. Если та же строка появилась снова в той же точке программы и других меняющихся данных нет, дальнейшая работа повторится: это цикл. Одинаковая строка в разных местах тела программы ещё не доказывает цикл.

Для машины Тьюринга повториться должны одновременно лента, положение и состояние. Повторение только состояния `q1` ничего не доказывает: оно нормально при проходе по длинной строке. Бесконечное движение по пустой ленте вправо может даже не повторять полную конфигурацию, потому что координата всё время меняется.

15. Как выбрать способ решения на экзамене

Сначала выпиши, что дано и что требуется. Для Редактора это исходная строка или ограничения на неё, точный порядок проверок и замен, условие остановки. Для машины Тьюринга - таблица, начальная позиция, начальное состояние, содержимое ленты и обозначение остановки.

Если строка короткая, можно выполнить программу вручную. Если строка длинная, но полностью известна, надёжно её воспроизвести. Если ищется неизвестное число в небольшом допустимом диапазоне, удобно перебрать его. Если данные огромны, нужна закономерность: повторяющаяся группа замен, обработка маркером, подсчёт изменений, однотипный проход по ленте.

При неизвестном порядке цифр проверь, влияет ли он на нужную величину. Для маркерного примера сумма от порядка не зависит; для `12 → 4` зависит число срабатываний. Здесь нужны доказательство независимости, точный перебор всех допустимых вариантов или математическая граница с примером достижения.

Перед ответом ещё раз прочитай вопрос. Просят строку, сумму, количество символов, исходное `n`, число шагов, минимальное значение или десятичное значение двоичной записи? Сама программа может быть выполнена правильно, но вывод не той величины испортит ответ.

Ошибка	Что проверять вместо привычки
Заменяются все вхождения	Одна команда Редактора меняет первое слева
Независимые `if` превращены в `elif`	Сохранена ли структура ветвления
Условие цикла проверено посреди тела	Где в программе стоит следующая проверка
Порядок цифр выбран без обоснования	Есть ли доказательство независимости
Остаток от деления принят за ответ	Доказаны ли повторение и малые случаи
При `S` пропущена запись	Выполнена ли команда перед остановкой
`N` принят за остановку	Что означает буква в условии
Головка начала на последней цифре	Не сказано ли «в пустой ячейке справа»
Прочитанная справа строка перевёрнута	Итог по-прежнему читается слева направо
Пустая клетка таблицы принята за стоп	Задана ли такая трактовка в условии
Максимум взят из случайной выборки	Проверены ли все варианты или доказана граница
Напечатан результат вместо параметра	Какая именно величина требуется

16. Практика: сначала реши самостоятельно

Здесь задания сгруппированы от базового чтения команд до применения закономерностей. Во всех задачах Редактора замена касается первого слева вхождения. В задачах про машину Тьюринга используются обозначения `L`, `R`, `N`, `S` из главы 10.

A. Точный смысл команд

1. Строка `11111`. Один раз выполняется `заменить(111, 2)`. Какая строка получится?

2. Строка `1212`. Сначала выполняется `заменить(12, 3)`, затем `заменить(32, 4)`. Какая строка получится?

3. На входе `111`. Тело программы: «ЕСЛИ нашлось(111), заменить(111, 2); КОНЕЦ ЕСЛИ; ЕСЛИ нашлось(2), заменить(2, 3); КОНЕЦ ЕСЛИ». Что получится? А если вторую проверку сделать веткой «ИНАЧЕ ЕСЛИ» первой?

4. На входе `12`. Выполняется цикл: «ПОКА нашлось(12): заменить(12, 3); заменить(3, 4)». Что получится и сколько будет проходов?

5. Для условия `нашлось(111) ИЛИ нашлось(33)` известна только конечная строка после завершения цикла. Может ли она содержать `33`? Может ли содержать `11`?

Б. Закономерности Редактора

6. Выполни программу главы 4 для восьми единиц. Запиши все строки после проходов и их количество.

7. Та же программа получает 30 единиц. Найди итоговую строку, сумму цифр и количество успешных замен.

8. Та же программа получает `n` единиц, `102 < n ≤ 200`. Найди минимальное `n`, при котором итоговая строка `1`.

9. В маркерном алгоритме главы 7 на вход подано `>111223`. Найди строку, сумму цифр и общую длину с маркером.

10. В программе `23 → 3332`, иначе `2 → 333` на вход подано `2` и четыре тройки. Найди количество троек, сумму цифр и число проходов.

11. Строка содержит 12 единиц и неизвестное число двоек в произвольном порядке. Выполняется `12 → 4`, пока возможно. Итоговая сумма равна 30. Найди минимальное число исходных двоек и построй подходящую строку.

12. Конечная строка из единиц и двоек не содержит `111` и `22` и содержит ровно три двойки. Сколько таких строк? Почему они все достижимы, если вход разрешён произвольный и цикл имеет условие `нашлось(111) ИЛИ нашлось(22)`?

В. Машина Тьюринга

13. Текущая ячейка содержит `0`, головка на координате 5 в состоянии `q1`. Выбрана команда `1, L, q2`. Что запишется, где окажется головка, какое будет состояние?

14. Та же исходная ситуация, команда `1, N, q2`. Завершится ли машина? А при команде `1, S, q2` что окажется в текущей ячейке?

15. Машина главы 11 получает `10100`, головка в пустой ячейке справа. Найди итог, конечную координату головки и число тактов. Координаты цифр начинаются с нуля.

16. Та же машина получает `000`. Найди результат и число тактов. Почему нельзя просто написать «минус один»?

17. У той же машины вход имеет длину 12, в результате ровно пять нулей. Найди минимум и максимум исходных нулей; приведи способы достижения.

18. Машина прошла двоичную запись числа 37 без изменений и дописала справа единицу. Какое десятичное число получилось?

19. Машина главы 13 формирует `10`, затем `k` единиц, затем `0`, где `k ≥ 1`. Найди наибольший результат не больше 500.

20. Машина один раз обрабатывает каждый символ, заменяя 5 и 7 на 1, а 9 на 0. В результате шесть символов, соседние различны. Найди минимальную исходную сумму.

17. Ответы с объяснениями

1. `211`. Заменяются первые три единицы; остальные две сохраняются.

2. `312`. Первая замена меняет только первые `12`. Цепочки `32` в новой строке нет, поэтому вторая команда ничего не меняет.

3. При независимых проверках - `3`, потому что вторая проверяет уже полученную двойку. При «ИНАЧЕ ЕСЛИ» - `2`: после первой подходящей ветви остальные в этой конструкции пропускаются.

4. `4`, один проход. Обе команды тела выполняются до повторной проверки условия.

5. `33` содержать не может: иначе условие с «ИЛИ» оставалось бы истинным. `11` может: это ещё не запрещённое `111`.

6. `311111 → 3311 → 111 → 3`. Четыре прохода.

7. Итог `311`, сумма 5. Группа из трёх замен сокращает чистый блок единиц на пять, пока их хотя бы шесть. Для 30 остаётся случай пяти единиц. Замен `111 → 3` будет 11, замен `33 → 1` будет 5, всего 16.

8. 106. Длины, дающие `1`, имеют вид `1 + 5m`, где `m` - целое неотрицательное. После 102 первое подходящее число - 106.

9. `222222335>`, сумма 23, общая длина 10. Шесть двоек, две тройки, пятёрка и маркер.

10. 15 троек, сумма 45, пять проходов. Четыре исходные тройки обрабатываются по одной, затем отдельный проход убирает двойку.

11. Шесть двоек. Подходит шесть повторений `12`, за которыми следуют шесть единиц. При числе двоек меньше шести итоговая сумма не превысит `12 + 3b < 30`.

12. 36. Два внутренних промежутка имеют по два варианта длины, два крайних по три. Каждую такую строку можно подать на вход без изменений: цикл не начнётся. Дополнительные ограничения на вход могли бы сделать этот довод неверным.

13. В клетке 5 появится единица, головка перейдёт на 4, состояние станет `q2`. Запись происходит до сдвига.

14. При `N` машина остаётся на координате 5 в `q2` и продолжает. При `S` в клетке 5 будет единица, затем машина остановится.

15. `10011`, координата 2, четыре такта. Остановочная запись заменяет единицу нулём и входит в число тактов.

16. `111`, пять тактов: вход, три замены, остановка на пустой ячейке. Для нулевого входа строка фиксированной длины становится набором единиц, а запись отрицательного числа алгоритмом не предусмотрена.

17. Минимум 4, максимум 11. Для минимума подойдут семь единиц, четыре нуля и последняя единица. Для максимума - четыре нуля, единица и семь нулей. В обоих случаях один итоговый ноль появляется из единицы.

18. 75, потому что дописывание единицы справа даёт `2 * 37 + 1`.

19. 382 при `k = 6`; при `k = 7` уже 766. Меньшие длины дают меньшие значения.

20. 42. В чередующейся строке чётной длины три единицы и три нуля. Для единиц выбираем исходные пятёрки, для нулей нужны девятки: `3 * 5 + 3 * 9 = 42`.

18. Как закреплять материал

Первый этап - научиться без ошибок выполнять одну команду и один проход. Достаточно коротких строк: они позволяют сразу увидеть неверно выбранное вхождение или пропущенную ветвь.

Второй этап - делать две независимые проверки одного примера: короткую ручную таблицу и программу. Их совпадение подтверждает правильность перевода команд. Однако программа не заменяет обоснование, если требуется перебрать неизвестный порядок цифр или доказать общий максимум.

Третий этап - после точного выполнения спрашивать, что повторяется, какая часть данных уже не изменится и что влияет на ответ. У Редактора это может быть длина однотипного блока, у машины Тьюринга - хвост нулей, длина двоичной записи или смысл состояния.

Для каждого типа полезно уметь своими словами объяснить результат без ссылки на запомненную формулу. Например: «Изменится хвост нулей и ближайшая единица слева от него» или «Маркер обрабатывает каждую исходную цифру ровно один раз». Такое объяснение позволяет адаптироваться, когда в условии поменяются цифры, направление движения или порядок проверок.

Источники для сверки определений и дальнейшей практики:

1. «Решу ЕГЭ», Исполнитель Редактор: https://inf-ege.sdamgia.ru/test?theme=356

2. «Решу ЕГЭ», Машина Тьюринга: https://inf-ege.sdamgia.ru/test?theme=426

3. Официальная документация Python, строки и replace: https://docs.python.org/3/library/stdtypes.html#str.replace

Материал сверён 15 октября 2026 года. Этот конспект разбирает два раздела по ссылкам пользователя; конкретный формат и обозначения экзаменационных заданий следует читать по самому условию.

Дополнение после исходного конспекта

Простой обратный подсчёт перед §12.2

Начнём обратные задачи с прохода, в котором каждый символ меняется независимо. Допустим, головка проходит непустую строку справа налево и заменяет 0 → 2, 1 → 0, 2 → 1. На пустой ячейке она останавливается. Порядок ячеек и длина строки сохраняются.

Исходный символ	Символ после прохода	Что можно восстановить по результату
0	2	Конечные двойки были нулями
1	0	Конечные нули были единицами
2	1	Конечные единицы были двойками

Например, исходная строка 111220 превращается в 000112. Если в результате три нуля, две единицы и одна двойка, исходно было три единицы, две двойки и один ноль. Чтобы ответить только о количествах, не требуется восстанавливать порядок всех символов. Это работает потому, что каждый символ обработан ровно один раз и его результат известен; для другой таблицы это нужно проверить заново.

Теперь перейдём к более сложной обратной задаче: у машины с хвостом нулей важно уже не только количество, но и расположение цифр. Для минимума или максимума понадобится и граница, и пример, который её достигает.

Маленькие проходы к §13: сторона приписывания, инверсия и перенос

Приписывание единицы слева сохраняет веса прежних цифр. Возьмём уже короткое двоичное слово 10101, то есть число 21. Цифры занимают ячейки 0–4, головка стоит в ближайшей пустой ячейке справа, на координате 5, начальное состояние q0.

Состояние	λ	0	1
q0	λ, L, q1	—	—
q1	1, L, q2	0, L, q1	1, L, q1
q2	λ, S, q2	—	—

Прочерки — незаданные команды: из указанного входа машина в эти сочетания не попадёт. Сначала выполним все шаги, продолжая читать ленту слева направо.

Выполнено тактов	Состояние после	Координата после	Что изменилось
0	q0	5	Исходная пустая ячейка справа
1	q1	4	Вход на последнюю цифру
2	q1	3	Последняя единица сохранена

Выполнено тактов	Состояние после	Координата после	Что изменилось
3	q1	2	Ноль сохранён
4	q1	1	Единица сохранена
5	q1	0	Ноль сохранён
6	q1	−1	Первая единица сохранена, достигнута пустота слева
7	q2	−2	В ячейке −1 появилась новая единица
8	q2, остановка	−2	Пустой символ сохранён, выполнена S

Получилось 110101, то есть 32 + 21 = 53. Для сравнения: единица справа дала бы 101011, то есть 2 · 21 + 1 = 43. Дописывание нуля справа даёт 101010, то есть 42. Сторона приписывания меняет разрядный смысл операции.

Почему так будет не только на этом примере? В q1 головка при каждом шаге сдвигается влево и сохраняет цифру. Поэтому уже пройденная правая часть остаётся прежней, а до левого края конечного слова остаётся всё меньше ячеек. Достигнув первой λ слева, машина добавляет единицу, делает один сдвиг и выполняет остановочную команду. Она обязательно завершится: по каждой из k цифр сделан один шаг, кроме них нужны вход, запись слева и остановка — всего k + 3 такта.

У исходных цифр сохраняются веса 1, 2, 4, … от правого края. Новая единица слева получает вес 2^k, поэтому для слова длины k результат равен $N + 2^k$. Это следует из разрядов, а не из имени состояния или одного удачного запуска. Например, для 101 получатся 1101 (13) и шесть тактов. Длину k нужно брать из самой строки: начальные нули тоже занимают ячейки.

При инверсии важно сохранить длину слова. Рассмотрим другой проход: каждую цифру машина заменяет противоположной (0 → 1, 1 → 0), проходит каждую ячейку ровно один раз и останавливается на λ. Из 101 получится 010: три символа, два нуля, но числовое значение 2. Если сразу заменить слово числом и записать его обратно как 10, для вопроса о количестве символов данные уже будут потеряны.

Для слова длины k исходная и инвертированная цифры в каждом разряде вместе дают единицу. Поэтому их числовые значения в сумме образуют k единиц: 101 + 010 = 111, то есть 5 + 2 = 7. Для любого такого слова значение после инверсии равно $2^k − 1 − N$. Когда длина неизвестна, одного конечного числового значения недостаточно для восстановления входа: нужно учитывать допустимую длину и начальные нули.

Прибавление единицы отличается от приписывания цифры. Пусть головка снова начинает в ближайшей λ справа от непустого двоичного слова, в q0. Теперь таблица другая:

Состояние	λ	0	1
q0	λ, L, q1	—	—
q1	1, S, q1	1, S, q1	0, L, q1

Для 1011 первый такт вводит головку на последнюю единицу. Следующие два такта дают 1010, затем 1000: единицы хвоста превращаются в нули, перенос идёт влево. На нуле записывается единица, и машина останавливается. Получилось 1100, то есть 11 + 1 = 12; всего четыре такта.

Если исходно были только единицы, например 111, машина проходит все три, получая 000, а затем на пустой ячейке слева пишет единицу и останавливается: итог 1000. Для этого входа пять тактов. Каждый пройденный бит хвоста уже обнулён; справа от головки лежит обработанная часть, слева остаётся перенос. Движение влево заканчивается на нуле или на первой пустой ячейке конечного слова, поэтому работа завершается. Как и для вычитания, арифметическое имя операции нужно выводить из конкретной таблицы.

Проверка новых объяснений

Для новых проходов проверь ещё четыре вопроса после объяснения:

1. В таблице приписывания слева замени исходное слово на 101. Какие слово, число и количество тактов получатся?

2. Почему несколько последовательных шагов в q1 этой таблицы не доказывают цикл, а q0 не является командой остановки?

3. После однократных замен 0 → 2, 1 → 0, 2 → 1 в результате четыре нуля, одна единица и две двойки. Сколько каждого символа было в начале?

4. Каковы слово, длина, число нулей и числовое значение после инверсии 1000?

Ответы с объяснениями:

1. 1101, число 13, шесть тактов: вход, три сохранённые цифры, запись слева, остановка.

2. Головка идёт по разным ячейкам к краю конечного слова: полная конфигурация не повторяется. q0 здесь начальное состояние, остановку задаёт S.

3. Два нуля, четыре единицы и одна двойка: каждый конечный символ однозначно указывает свой исходный.

4. 0111, четыре символа, один ноль, значение 7. Ведущий ноль остаётся ячейкой ленты.